Rapport SAÉ 302
Développer des applications communicantes

Contexte **3**
Objectifs **3**
Organisation **3**
NetBeans **4**
Qu’est-ce que NetBeans? 4
Pourquoi l’utiliser? 4
Présentation de notre serveur 4
Présentation de notre client graphique 4
Android Studio **12**
C’est quoi Android Studio? 12
Pourquoi l’utiliser? 12
Architecture Technique - Partie Client (Android Studio) 12
Le Noyau Réseau 13
Les Contrôleurs d'Écran (Activity et Fragments) 14
Les Adaptateurs et Modèles 14
Les Layouts et Drawables (XML) 14
Fonctionnalités Clés 15
Messagerie Privée (L'envoi) 15
Système de Notification en point rouge 15
Détection Automatique des Conversations 15
L'affichage dynamique des bulles (ChatAdapter.java) 16
Présentation de notre application 16

Contexte

Au cours de cette saé, nous avons créé une application de messagerie de type WhatsApp avec Android Studio et NetBeans permettant à deux utilisateurs de discuter entre eux ainsi qu'entre membres d'un groupe.

Objectifs

Au cours de cette saé, nous avons eu pour objectif de :

- Réaliser une application de réseau social
    - Gestion des utilisateurs
    - Demande d'ami
    - Constitution de groupes de discussion
- Élaborer un protocole applicatif avec architecture client/serveur
- Langage : Java, Netbeans/swing et Android Studio
- Réaliser un serveur et un client test
- Conception de type MVC et réalisation d'un ou 2 clients graphiques
- Intégrer un protocole client et serveur

Organisation

Joyez Florian :

- Chef de projet
- Développeur client Android Studio

Barkani Mohamed-Amin:

- Développeur client Android Studio

Azmani Redouane:

- Développeur client NetBeans
- Responsable serveur NetBeans

NetBeans

Qu’est-ce que NetBeans?

Apache Netbeans est un logiciel qui permet de développer des applications en Java. Il est possible d’utiliser plusieurs langages dans NetBeans mais il est reconnu pour l’utilisation de java. C’est beaucoup plus simple et intuitif qu’Android Studio, par exemple.

Pourquoi l’utiliser?

Nous l’avons utilisé pour coder un serveur, un client scénario et un client graphique pour faire des tests en prévision de l’application de réseau social finale sur Android Studio.

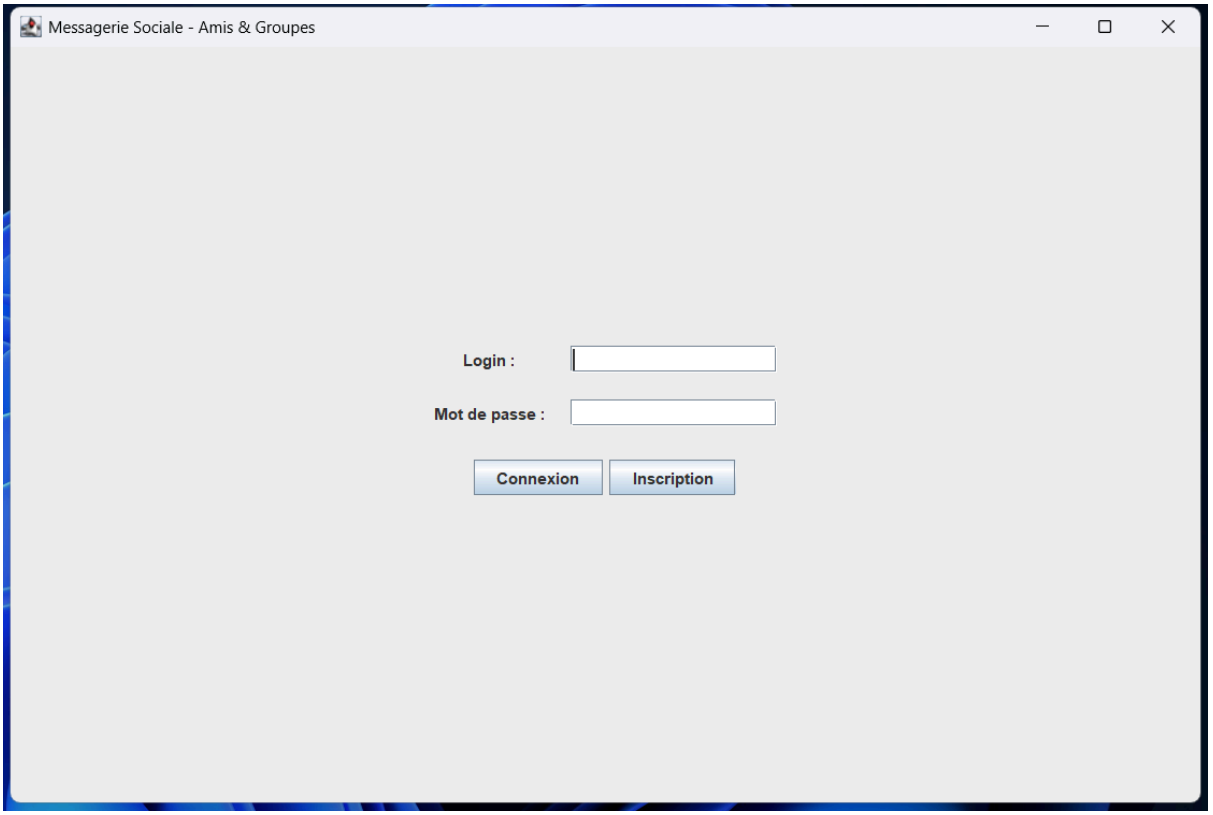
Présentation de notre serveur

Tout d’abord, le serveur démarre sur le port 3333 car les ports en dessous de 1234 peuvent être bloquants. Il attend ensuite une réponse du client, quand le client aura envoyé une réponse, il triera cette réponse en fonction du message envoyé, si c’est une inscription, une demande d’ami ou une création de groupe par exemple. Il permet de garder en mémoire des informations comme le nom d’utilisateur ou les messages.
Cependant, quand il y a un envoi de message d’Alex vers Hakim, le serveur vient intercepter le message d’Alex et le stocke en attendant que Hakim ouvre sa messagerie pour enfin voir son message.

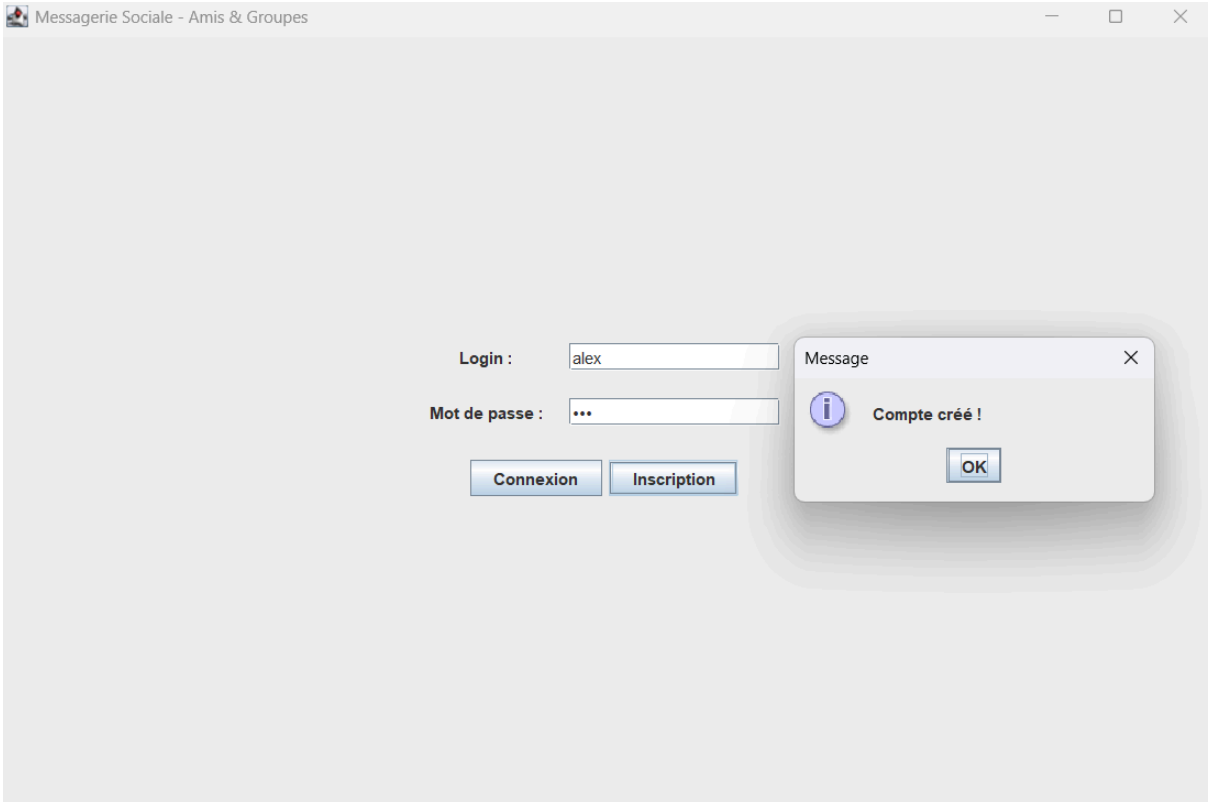
Présentation de notre client graphique

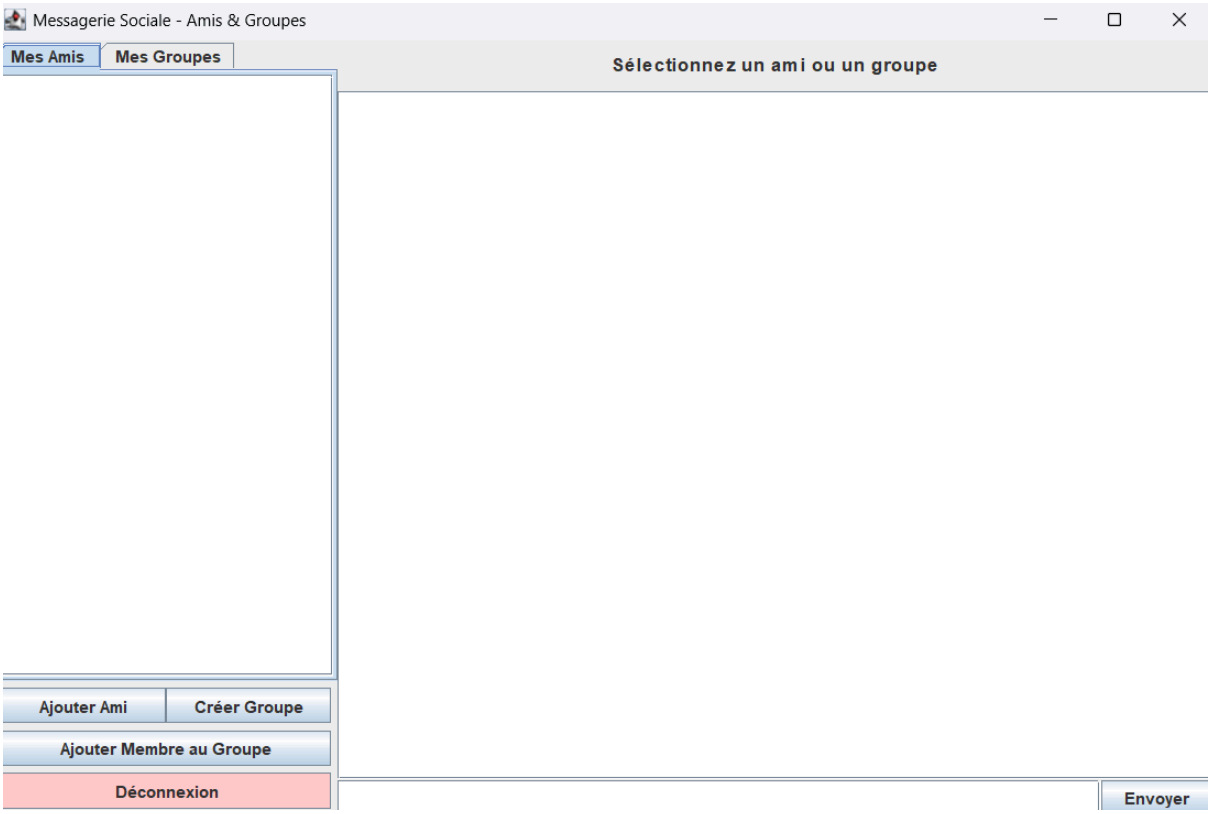
Le client graphique est la seule chose que verra un utilisateur lambda. Il ne saura pas comment fonctionne le serveur, il n’aura qu’à appuyer sur des boutons et saisir des informations.
Sur le client, l’utilisateur écrira un message “Salut, ça va ?” et le transformera pour que ce soit interprété par le serveur afin de connaître l’émetteur, le destinataire et le contenu du message.
Le client ne cessera d’envoyer des demandes au serveur pour savoir s’il y a du nouveau contenu à afficher pour ses utilisateurs. Ensuite, quand le serveur lui aura envoyé une réponse il devra la “dessiner” pour l’afficher correctement aux utilisateurs. Il transforme les messages du serveur pour les rendre plus lisibles visuellement parlant.

Ceci est la page de connexion ou d'inscription, c’est là que l’utilisateur pourra soit créer un nouveau compte ou bien se connecter.

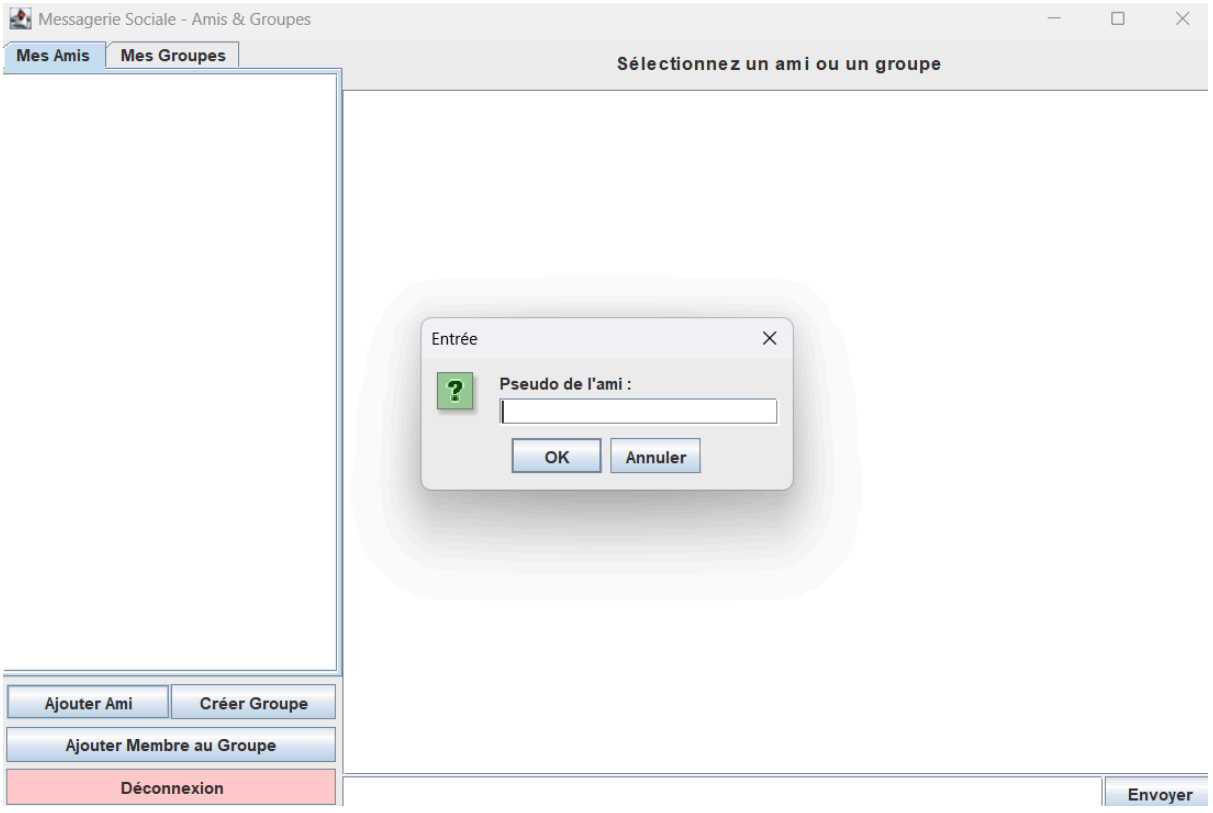


Ici l’utilisateur “alex” crée un nouveau compte, elle peut maintenant se connecter.
Une fenêtre pop-up s'ouvre pour l’en informer.

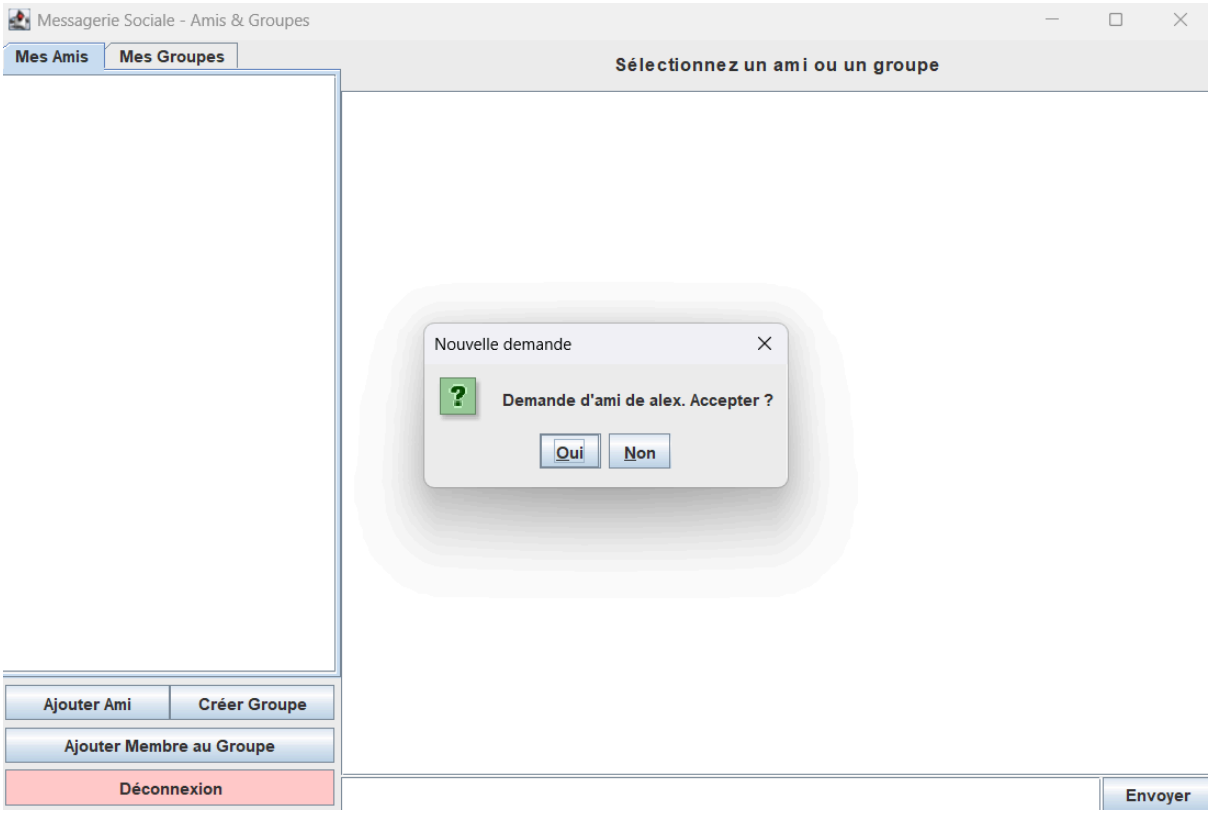




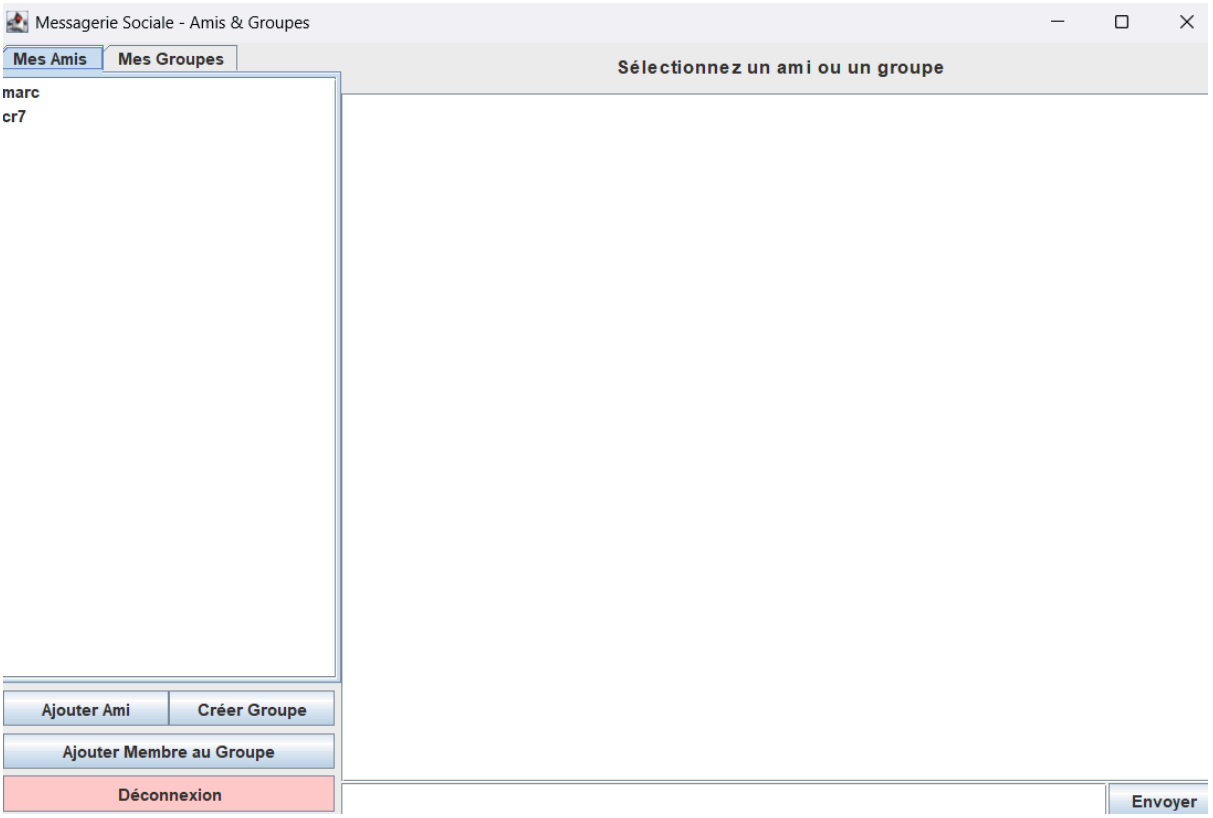
Alex peut désormais interagir avec l'interface, elle peut maintenant ajouter un ami ce qu'elle décide de faire en premier. Elle doit maintenant entrer le nom d'utilisateur de cet ami.



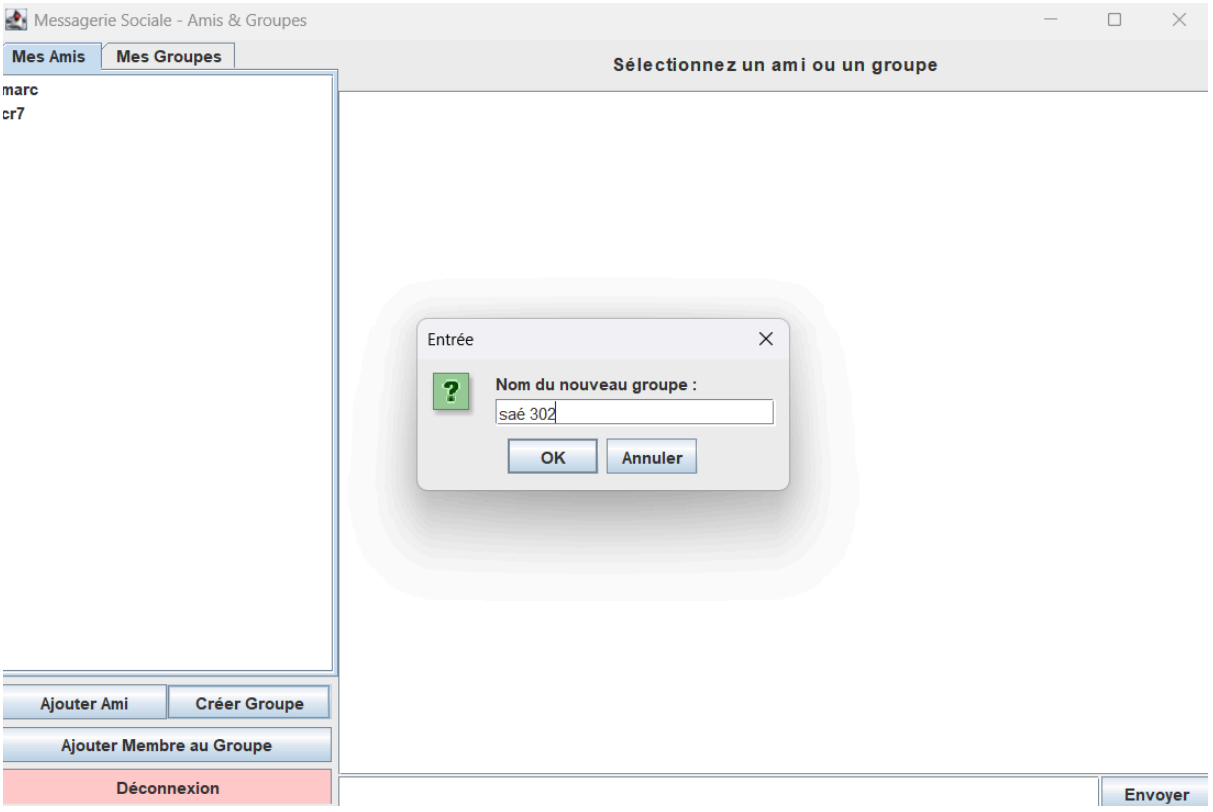
Cet ami aura une notification pour accepter la demande lorsqu'il se connectera, il pourra soit l'accepter soit la refuser.

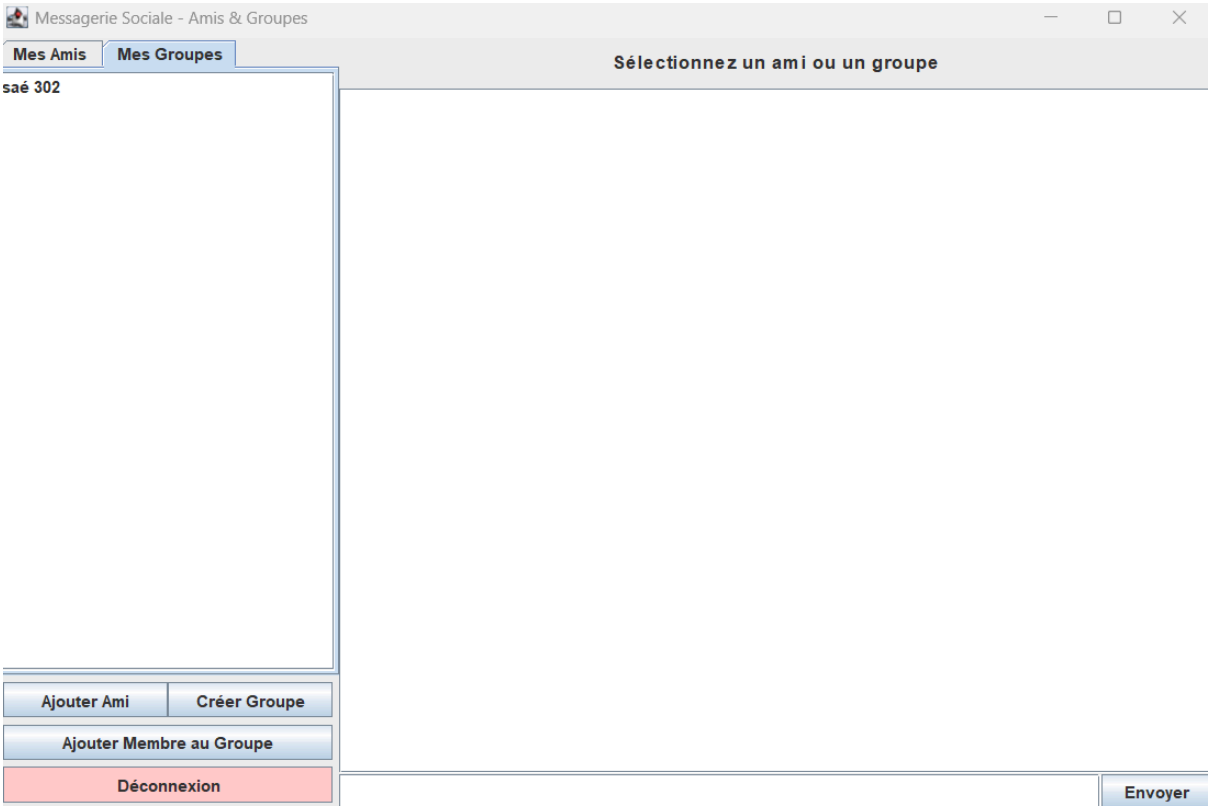


Les différents amis s'afficheront dans une colonne à gauche, on peut maintenant cliquer sur un ami pour entamer une conversation privée avec lui.

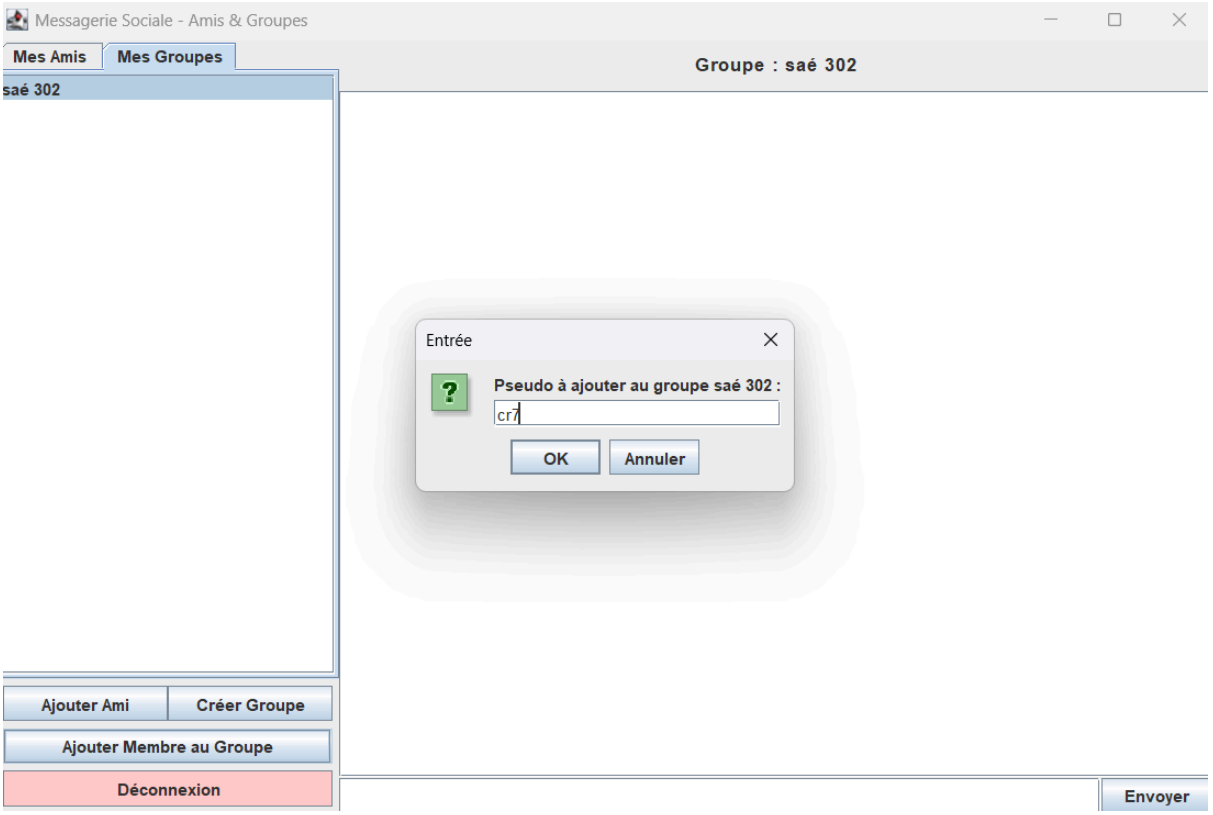


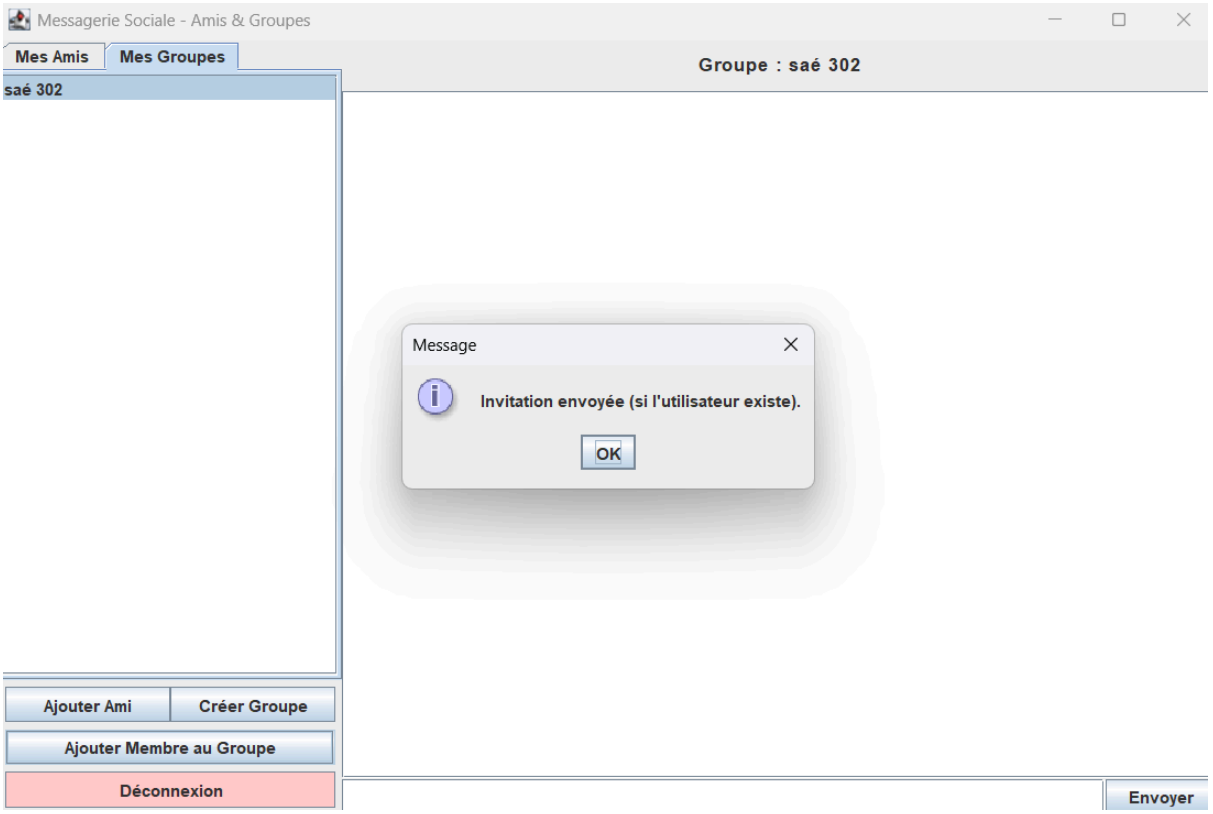
Il y a aussi la possibilité de créer un groupe et de lui donner un nom.





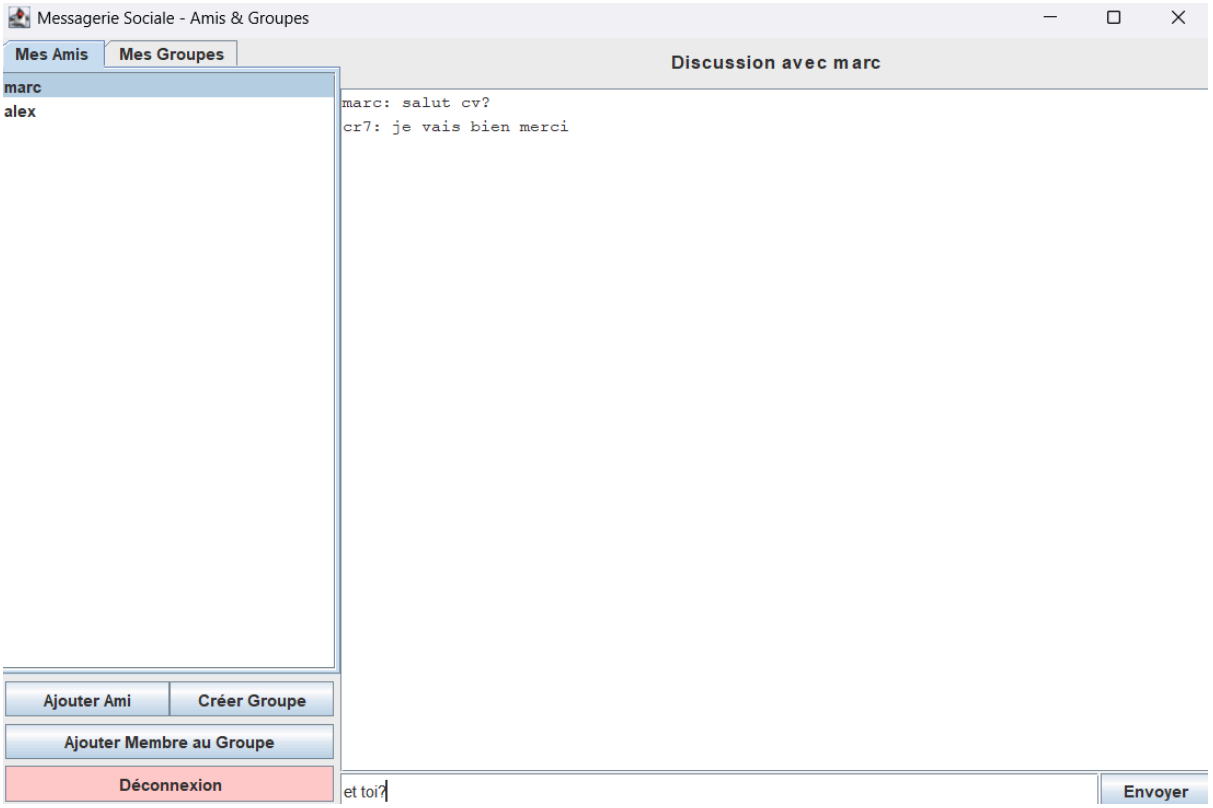
Nous pouvons maintenant ajouter des membres aux groupes qui seront dans notre liste d'amis en cliquant sur le bouton en bas à gauche “Ajouter Membre au Groupe”.





L'utilisateur ayant reçu la demande sera automatiquement ajouté au groupe et pourra envoyer et voir les messages des différents utilisateurs du groupe.

Ici nous avons un exemple de conversation privée entre deux utilisateurs, les messages s'affichent sous la forme "pseudo : contenu du message" pour savoir à qui a écrit le message qui a été envoyé.

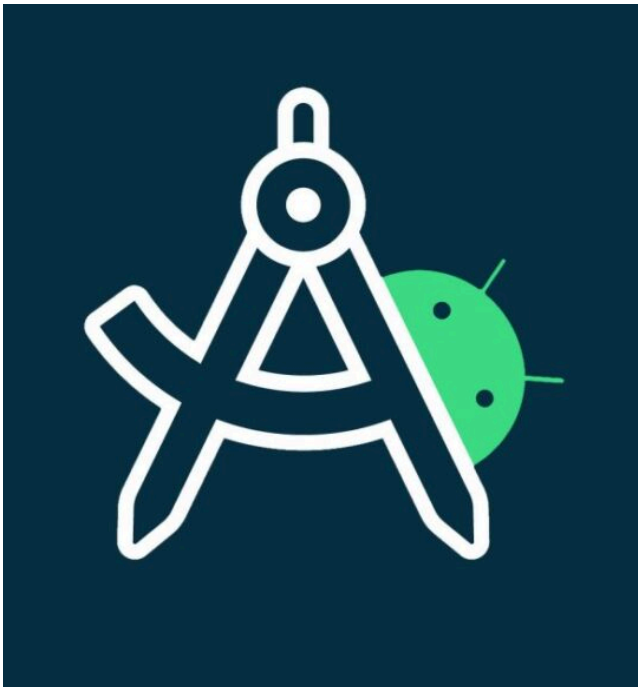


Sécurisation des messages entre amis : seules les personnes qu'on a ajoutées en ami peuvent nous envoyer des messages.

Pour les clients graphiques sur NetBeans, il y a la possibilité d'envoyer des messages seulement aux amis ou bien à une personne qu'on n'a pas en amis seulement dans un groupe car on peut ajouter n'importe quel utilisateur dans un groupe. On ne peut pas supprimer les amis et les groupes. On peut aussi créer un compte, se déconnecter mais pas supprimer le compte.

Android Studio

C’est quoi Android Studio?



Android Studio est un environnement de développement développé par Google en 2014 pour la création d’applications mobiles Android. Android Studio propose un développement d'applications dans le langage Java et Kotlin.

Pourquoi l’utiliser?

Android Studio est une application intéressante car elle permet de développer des applications en ayant la certitude de leur compatibilité. De plus, cette application permet d’avoir des téléphones virtuels pour simuler l’application.

Architecture Technique - Partie Client (Android Studio)

L'application est structurée selon le modèle MVC adapté à Android, séparant clairement la logique, les données et l'apparence.

Le Noyau Réseau

- UdpClient.java :
  C'est la classe utilitaire critique de l'application. Android interdisant les opérations réseau sur le fil principal (Main Thread) pour éviter de figer l'écran, cette classe gère l'envoi des paquets UDP dans un Thread secondaire. Elle utilise une interface (UdpListener) pour "rappeler" l'activité et lui fournir la réponse du serveur une fois reçue. On a dû utiliser ce thread secondaire pour ne pas faire planter l'application lors de l'envoi.

  Voici le code pour cela avec commentaires fournis

```
// Méthode pour envoyer et recevoir sans figer l'écran

public static void sendAndReceive(final String message, final
UdpListener listener) {new Thread(new Runnable() {   @Override

 public void run(

 try {         // Création du socket et envoi

    DatagramSocket socket = new DatagramSocket()
InetAddress serverAddr = InetAddress.getByName(SERVER_IP);
    byte[] sendData = message.getBytes();

  // ... envoi du paquet ...  // puis attente de la réponse du
serveur

       byte[] buffer = new byte[4096];
DatagramPacket receivePacket = new DatagramPacket(buffer,
buffer.length); socket.receive(receivePacket);

         // On renvoie la réponse à l'activité

               if (listener != null) {

   listener.onReceive(new String(receivePacket.getData()).trim());

         }

     } catch (Exception e) { e.printStackTrace(); }

      }

    }).start();

}
```

Les Contrôleurs d'Écran (Activity et Fragments)

Les Activity.java gèrent les écrans complets, tandis que les Fragments.java gèrent les onglets à l'intérieur de l'écran principal.

MainActivity.java **:** C'est le template principal. Elle gère le menu latéral , l'affichage du nom de l'utilisateur dans l'en-tête, et la navigation entre les fragments.

LoginActivity et RegisterActivity.java **:** Ils gèrent les formulaires d'entrée et la connexion/inscription

PrivateChatActivity.java **:** il gère l'écran de conversation entre 2 amis. Elle lance une boucle de rafraîchissement toutes les secondes pour récupérer l'historique et met à jour l'affichage via l'adaptateur.

MessagesFragment.java **:** Contient la logique la plus complexe (tache de fond). Il vérifie toutes les 3 secondes si des amis ont envoyé des messages. Elle compare la taille de l'historique serveur avec celle mémorisée localement pour décider d'afficher ou non le point rouge de notification sur une conversation

AmisFragment.java : Gère l'affichage de la liste d'amis et les interactions sociales (inviter, accepter des demandes).

GroupesFragment.java : Affiche la liste des salons de discussion disponibles pour l'utilisateur.

AccueilFragment.java **:** Affiche la page de bienvenue personnalisée.

Les Adaptateurs et Modèles

Les adaptateurs servent de pont entre les données Java et l'interface XML

-ChatAdapter **:** C'est le responsable du rendu des bulles de discussion. Équipé d'une Logique dynamique comme dans la méthode getView, il analyse l'objet MessageChat. Si le message vient de l'utilisateur courant , il aligne le texte à droite avec un fond coloré. Sinon, il l'aligne à gauche avec un fond gris.

-ConversationsAdapter : Gère la liste des discussions actives. Il gère la visibilité du badge de notification et l'interaction clic long pour supprimer une conversation.

-MessageChat : Classe modèle simple stockant le texte du message.

-GroupesAdapter : Gère l'affichage de la liste des noms de groupes.

Les Layouts et Drawables (XML)

Pour offrir une interface moderne, nous n'utilisons pas les composants par défaut bruts, mais des formes vectorielles définies en XML dans le dossier res/drawable

- bulle_moi_bg.xml / bulle_ami_bg.xml **:** Définissent les formes des messages avec des coins arrondis asymétriques pour créer l'effet de "bulle de BD"
- badge_notif.xml **:** Définit un cercle rouge pur pour les notifications.-
- fond_accueil_violet.xml : Définit une sorte de rectangle violet avec des bords arrondis pour la carte de bienvenue sur la page d'accueil.
- background_input.xml : Il stylise la zone de saisie de texte dans les conversations privées entre amis avec des bords ronds et une couleur de fond légère pour donner un style comme iMessage dans Iphone.

Fonctionnalités Clés

Cette section explique comment s'est passé l'implémentation des fonctionnalités majeures de l'application SAE302.

Messagerie Privée (L'envoi)

L'envoi est asynchrone grâce à notre classe PrivateChatActivity.java

```
// Construction de la commande CSV et envoi avec thread séparé
String cmd = "envoyer_msg_prive," + monLogin + "," + destinataire
+ "," + msg;
UdpClient.sendAndReceive(cmd, null);
```

Système de Notification en point rouge

Un algorithme de comparaison détermine si une notification doit être affichée.

On le fera dans MessagesFragment.java

```
// Si la taille serveur est plus grande que la  taille mémorisée
localement
if (tailleActuelle > dernieretailleConnue) {
    // Et que le dernier message ne vient pas de moi     if
(!dernierMessage.startsWith(monLogin + ":")) {
        // Active le point rouge dans la liste
        unreadSet.add(ami);
        adapter.updateNotifications(unreadSet);
    }
}
```

Détection Automatique des Conversations

Une boucle infinie vérifie la présence de nouveaux messages périodiquement toutes les 3 secondes.

La voici dans MessagesFragment.java

```
private void lancerDetectionMessages() {
    runnableVerification = new Runnable() {
        @Override
        public void run() {
            // Vérifie l'historique pour chaque ami
            verifierHistoriqueAvec(amiPotentiel);
            // Relance la vérification dans 3 secondes
            handler.postDelayed(this, 3000);
        }
    };
    handler.post(runnableVerification);
}
```

L'affichage dynamique des bulles (`ChatAdapter.java`)

C'est dans cette classe qu'on décide visuellement si le message est à droite ou à gauche.

```
@Override
public View getView(int position, View convertView, ViewGroup
parent) {
    MessageChat message = getItem(position);

    // Si c'est moi qui ai envoyé le message
    if (message.isEstMoi()) {
        holder.layoutMoi.setVisibility(View.VISIBLE); // Affiche
bulle droite
        holder.layoutAmi.setVisibility(View.GONE);    // Cache
bulle gauche
        holder.tvMoi.setText(message.getTexte());
    }
    // Si c'est mon ami qui parle
    else {
        holder.layoutMoi.setVisibility(View.GONE);
        holder.layoutAmi.setVisibility(View.VISIBLE); // Affiche
bulle gauche
        holder.tvAmi.setText(message.getTexte());
    }
    return convertView;
}
```

Présentation de notre application

Pour le lancement de notre application, nous avons développé des pages permettant à l'utilisateur de se connecter à son compte mais aussi une page pour qu'il puisse le créer.



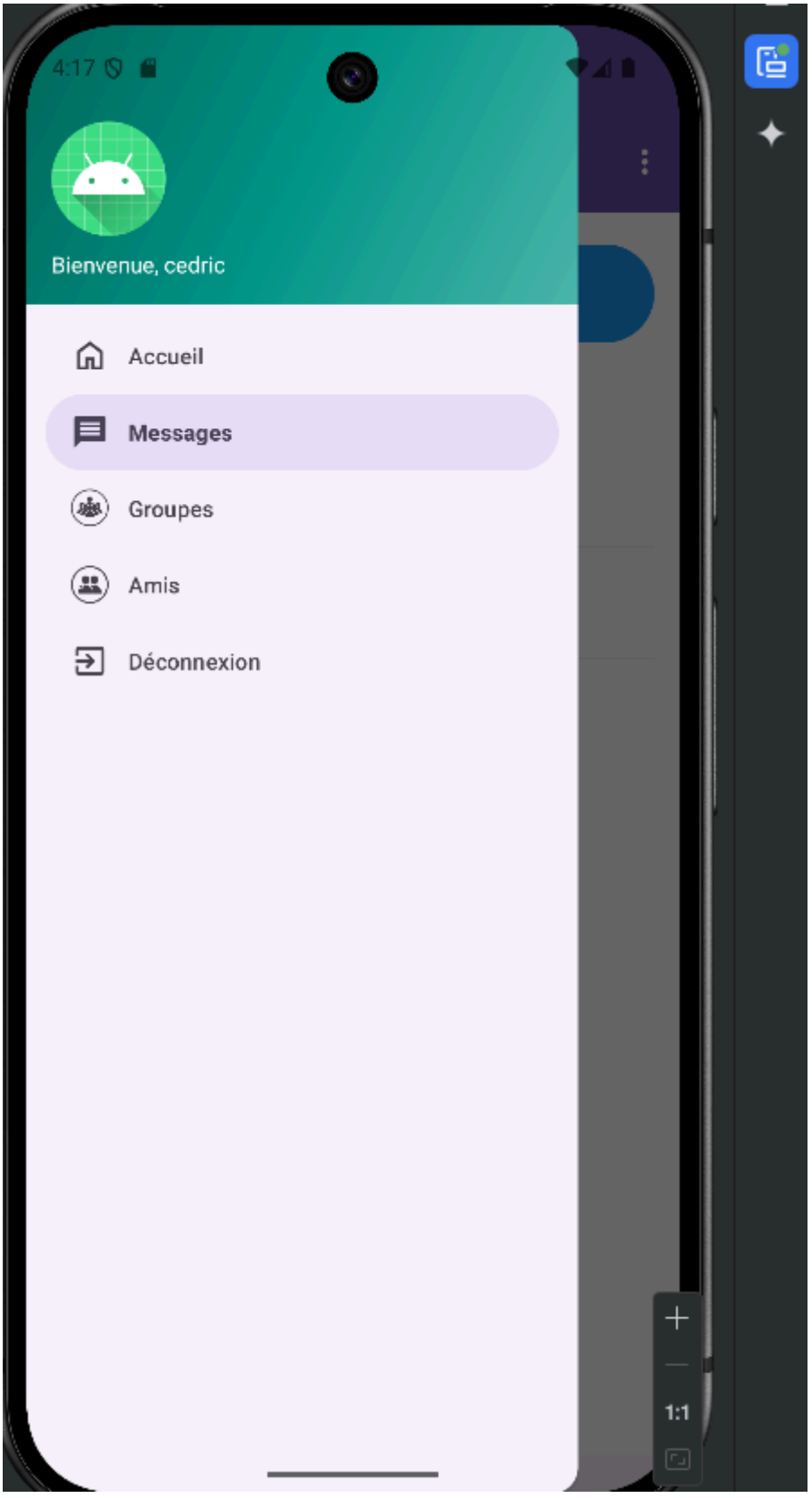


Après s'être connecté, l'utilisateur arrive au niveau de la page d'accueil où un message souhaite la bienvenue à l'utilisateur.

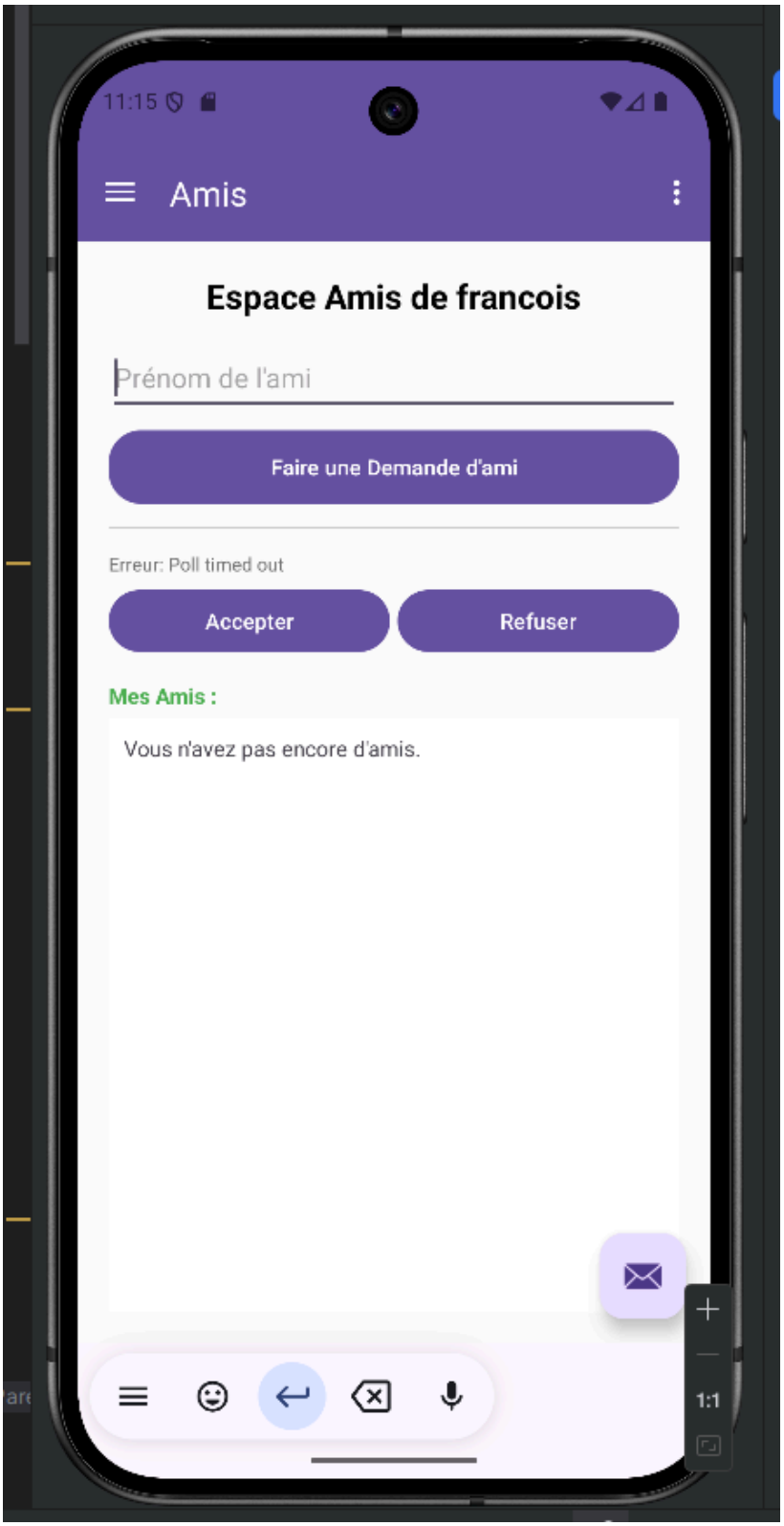


De cette page, il est par la suite possible d’ouvrir un menu déroulant (drawer) au niveau des 3 petites barres horizontales en haut à gauche de l'écran pour pouvoir passer d’une page à

une autre et accéder aux fonctionnalités de l'application ; messages, groupes, amis et déconnexion.



Voici la page amis qui permet à l'utilisateur d'ajouter des utilisateurs à sa liste d'amis, il peut accepter ou refuser des demandes d’ami de la part d’autres utilisateurs et voir le nom de tous ses amis.



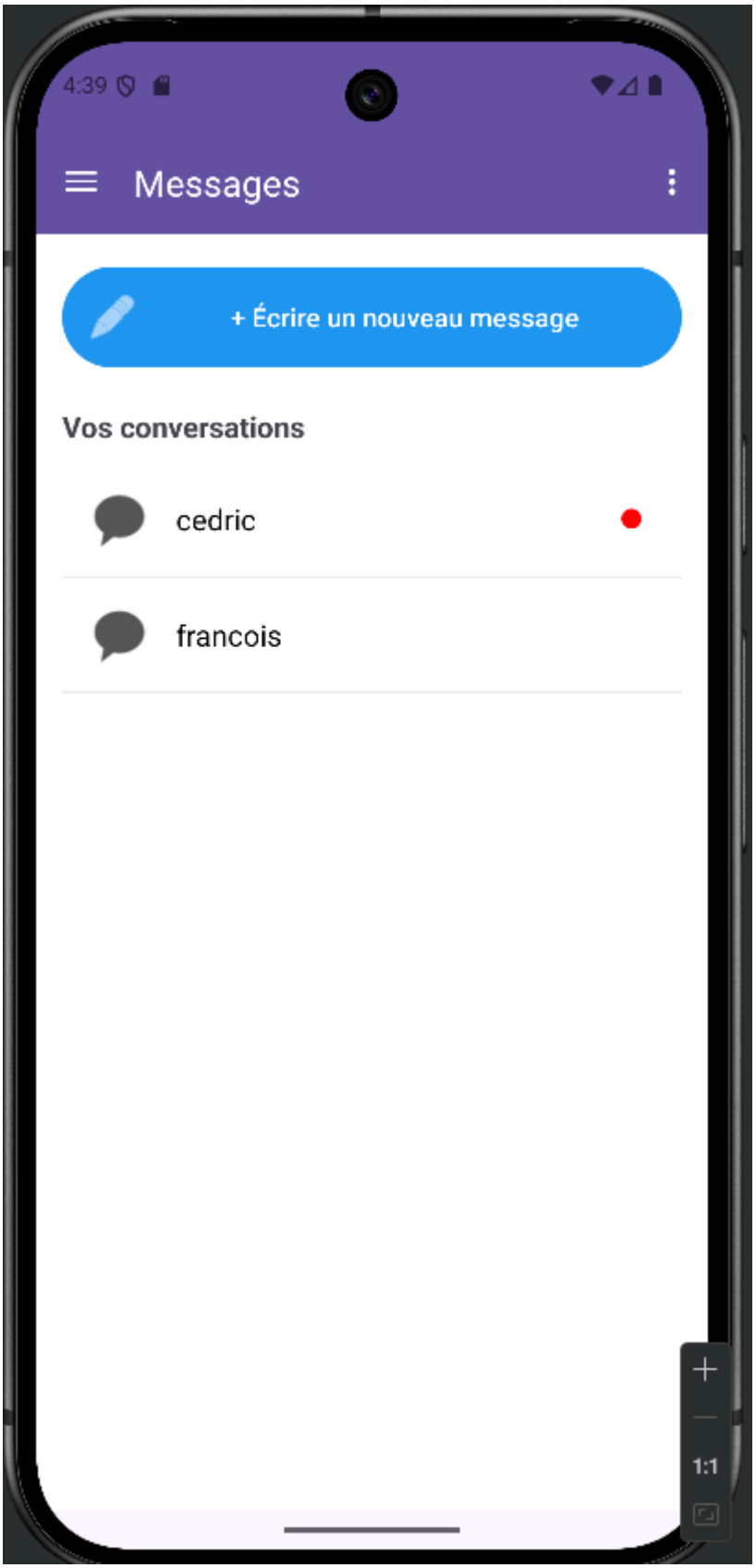
Voici la page message qui permet à l'utilisateur d'envoyer des messages à ses amis. Comme vous pouvez le voir, il faut dans un premier temps choisir avec lequel de vos amis vous voulez discuter.



Ensuite, sur la nouvelle page qui apparaît, il est possible d’envoyer des messages à cette personne.

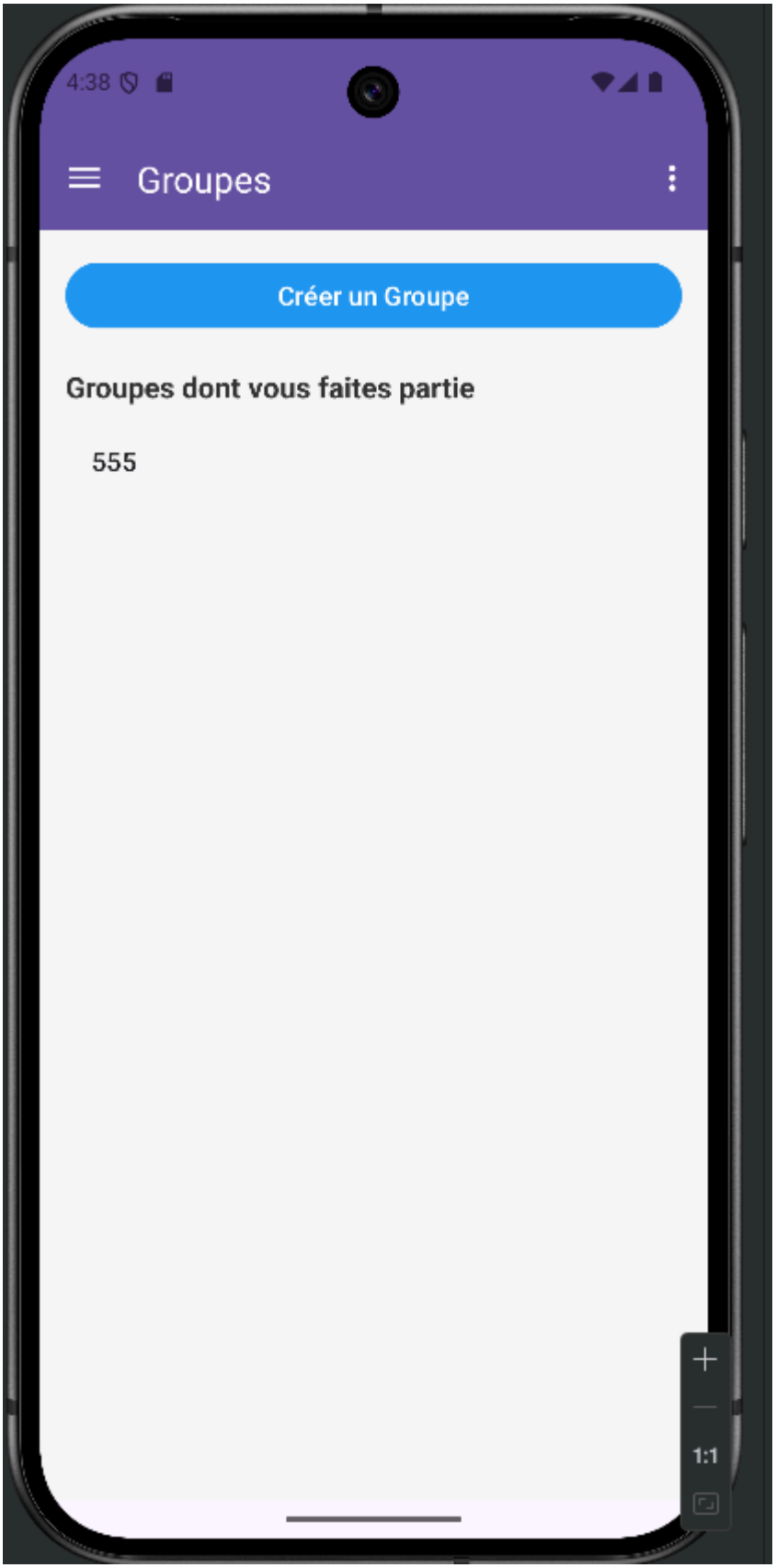


Lorsqu’un utilisateur envoie un message à une personne, le destinataire peut apercevoir un petit point rouge au niveau de la discussion pour l’informer qu’il vient de recevoir un nouveau message.

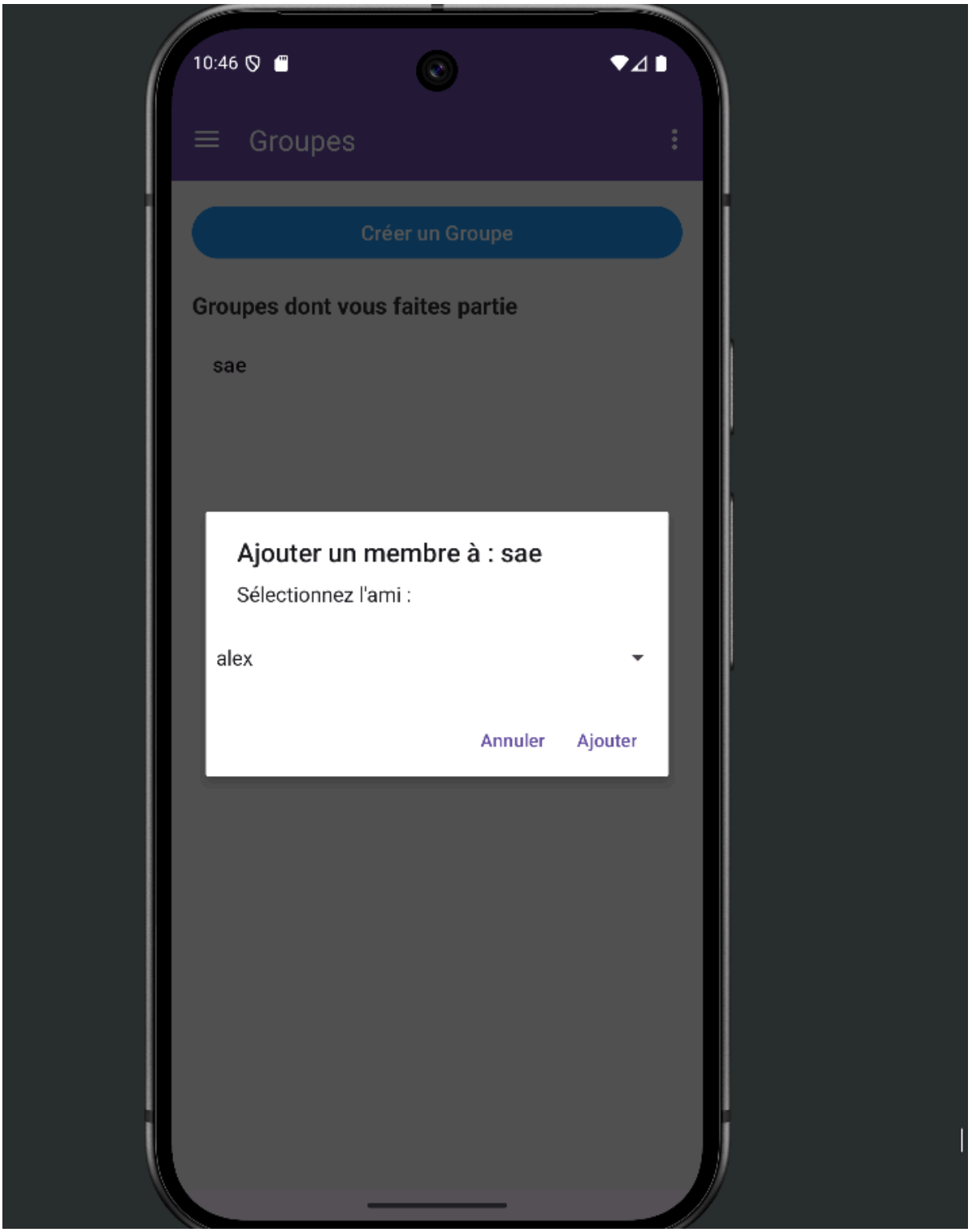


Dans notre application, il est également possible de créer des groupes avec plusieurs personnes. Pour le faire, il est nécessaire d'appuyer sur le bouton "créer un groupe" et de lui mettre un nom.

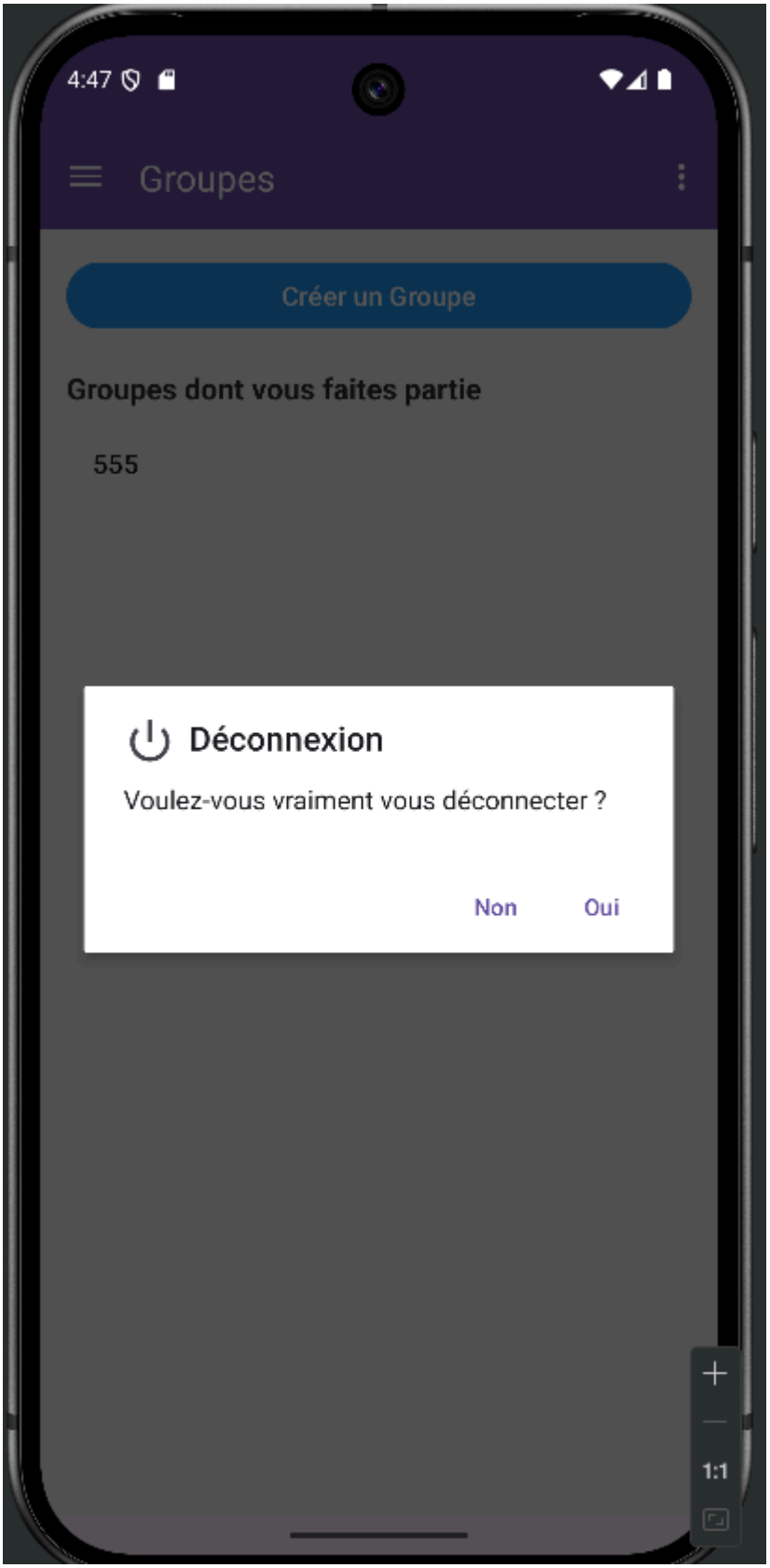



4:38
Groupes
Créer un Groupe
Groupes dont vous faites partie
555
1:1

Après la création du groupe, pour ajouter des personnes à l'intérieur, il faut appuyer pendant quelques secondes sur le nom du groupe. Cette action fera apparaître le pop-up permettant d'ajouter les personnes souhaitées aux groupes



Pour permettre la déconnexion, nous avons créé un bouton de déconnexion. Cependant, pour éviter les déconnexions accidentelles, nous avons fait en sorte qu'un pop-up apparaisse pour confirmer l'intention réelle de l'utilisateur.


4:47
Groupes
Créer un Groupe
Groupes dont vous faites partie
555
Déconnexion
Voulez-vous vraiment vous déconnecter ?
Non
Oui
1:1